

Vendredi 27/02/2026

Rapport Back-end Partie : Notifications

Le dossier notifications gère le système de messagerie et de notifications du projet ShareTrajet.

Il permet :

- ***d'envoyer des messages entre utilisateurs***
- ***de créer des conversations***
- ***d'envoyer des notifications***
- ***d'afficher les notifications en temps réel via WebSocket***
- ***de gérer les événements automatiquement grâce aux signals***

Dossier migrations

Il contient les fichiers qui permettent à Django de créer et modifier les tables dans la base de données automatiquement.

Autrement dit, ces fichiers décrivent la structure des tables du projet (messages, conversations, notifications...), il contient les fichiers suivants :

__init__.py :

Ce fichier est vide, mais il est nécessaire pour indiquer à Python que le dossier migrations est un module Python.

Cela permet à Django de reconnaître et charger les migrations.

0001_initial.py :

C'est la première migration du module notifications.

Ce fichier sert à créer les tables principales de la messagerie et des notifications dans la base de données.

Tables créées : Il crée principalement trois modèles :

1. Conversation

Table qui représente une discussion entre utilisateurs.

Elle contient :

- les participants
- le trajet associé (optionnel)
- le dernier message
- la date de création
- la dernière activité

Cela permet d'organiser les messages par conversation.

2. Message

Table qui stocke tous les messages envoyés entre les utilisateurs.

Chaque message contient :

- l'expéditeur
- le destinataire
- le texte du message
- un fichier média (image ou pdf)
- le trajet associé
- la conversation
- l'état de lecture

Elle permet donc de gérer la messagerie du système ShareTrajet.

3. *Notification*

Table qui stocke les notifications envoyées aux utilisateurs.

Par exemple :

- nouvelle réservation
- message reçu
- trajet annulé
- réservation acceptée

Chaque notification contient :

- l'utilisateur qui la reçoit
- l'utilisateur qui l'a envoyée
- le type de notification
- le contenu
- la date
- si elle est lue ou non

Index ajoutés

Le fichier ajoute aussi des index pour améliorer les performances de la base de données lors des recherches.

Exemple :

- recherche des messages non lus
- recherche des notifications
- tri par date

0002_alter_notification_related_model.py

Cette migration modifie légèrement la table Notification.

Elle rend le champ :related_model

optionnel (null=True)

Cela permet de créer une notification sans obligatoirement associer un objet spécifique.

Fichiers à l'extérieur du dossier migrations

1.__init__.py

Fichier vide qui indique que le dossier notifications est un module Python.

Cela permet à Django d'importer correctement les fichiers de cette application.

2.admin.py

Ce fichier sert à configurer l'interface d'administration Django pour les notifications.

Grâce à ce fichier, l'administrateur peut :

- voir les notifications
- les filtrer
- les rechercher
- les marquer comme lues ou non lues

Fonctionnalités ajoutées

Dans l'interface admin on peut :

- afficher : id , destinataire , type , aperçu du contenu , statut de lecture , date
- filtrer par : type , statut , date
- actions rapides : marquer comme lues ou non lues

3. apps.py

Ce fichier configure l'application Django notifications.

Il permet de :

- définir le nom de l'application
- charger automatiquement les signals

La méthode : ready() charge automatiquement :

app.notifications.signals

Donc dès que le serveur démarre, les signals sont activés.

4. consumers.py

Ce fichier gère les WebSockets pour les notifications en temps réel.

Permet d'envoyer les notifications instantanément aux utilisateurs connectés sans recharger la page.

Technologie utilisée :

- Django Channels
- WebSocket

Fonctionnalités :

Lorsqu'un utilisateur se connecte :

- connexion WebSocket
- récupération des notifications non lues

- envoi des notifications au navigateur

L'utilisateur peut aussi :

- marquer une notification comme lue
- marquer toutes les notifications comme lues

5.models.py

Ce fichier définit les modèles de base de données du système de messagerie et de notifications.

Il contient trois modèles principaux.

Message : représente un message envoyé entre utilisateurs.

Contient :

- sender (expéditeur)
- receiver (destinataire)
- texte
- média
- trajet associé
- statut de lecture
- date

Il permet de gérer la messagerie privée et de groupe.

Conversation : Représente une discussion entre plusieurs utilisateurs.

Elle contient :

- les participants
- le trajet associé

- le dernier message
- la date de dernière activité

Cela permet d'organiser les messages par discussion.

Notification : Représente une notification envoyée à un utilisateur.

Exemples :

- message reçu
- réservation demandée
- réservation acceptée
- trajet annulé

Chaque notification contient :

- le destinataire
- l'expéditeur
- le type
- le contenu
- l'objet concerné
- la date
- le statut (lu ou non)

6. routing.py

Ce fichier définit les routes WebSocket utilisées par Django Channels.

Il permet de connecter le frontend avec le backend pour :

Notifications

ws/notifications/

Chat de groupe

ws/chat/group/<trajet_id>/

Chat privé

ws/chat/private/<conversation_id>/

Ces routes permettent au frontend de recevoir les messages et notifications en temps réel.

7. serializers.py

Ce fichier transforme les objets Django en données JSON pour les envoyer via l'API.

Il contient le serializer :

NotificationSerializer : Convertir une notification en données JSON envoyées au frontend.

Exemple de données envoyées :

- id
- type
- contenu
- expéditeur
- date
- statut de lecture

8. signals.py

Ce fichier contient les signals Django.

Les signaux permettent d'exécuter automatiquement du code lorsqu'un événement se produit dans la base de données.

Exemple 1 : création d'un utilisateur

Quand un utilisateur est créé :

➡ une notification de bienvenue est envoyée.

Exemple 2 : envoi d'un message

Quand un message est créé :

➡ une notification est envoyée au destinataire.

Pour :

- messages privés
- messages de groupe

9.urls.py

Ce fichier définit les routes API REST pour les notifications, Il utilise : Django Rest Framework Router

Cela permet d'accéder aux endpoints comme :

/notifications/
/notifications/{id}

pour :

- voir les notifications
- les modifier
- les marquer comme lues

10.utils.py

Ce fichier contient les fonctions utilitaires pour créer et envoyer des notifications.

Il centralise la logique de notification.

Fonction principale :

`create_notification()`

Elle fait deux choses :

- crée la notification dans la base de données
- l'envoie en temps réel via WebSocket

Fonctions spécialisées :

`notify_new_message`

notification lorsqu'un message est reçu

`notify_reservation_request`

notification lorsqu'une réservation est demandée

`notify_reservation_approved`

notification lorsqu'une réservation est acceptée

`notify_reservation_rejected`

notification lorsqu'une réservation est refusée

11.Views.py

Ce fichier contient les vues de l'API REST qui permettent de gérer les notifications dans le backend du projet ShareTrajet.

Il définit un ViewSet (souvent NotificationViewSet) utilisant Django REST Framework, qui permet aux utilisateurs de :

- récupérer la liste de leurs notifications,
- voir le détail d'une notification,
- marquer une notification comme lue,
- marquer toutes les notifications comme lues.

Ces vues communiquent avec le modèle `Notification` et utilisent le `NotificationSerializer` pour transformer les données de la base de données en format JSON afin qu'elles puissent être utilisées par le frontend de l'application.

Ainsi, ce fichier joue un rôle important dans la communication entre le frontend et le système de notifications du backend.

Oudjane Anias 2CP7